10-02-2026

Lab - 12

Section 1: PyTorch

Import Torch

In [ ]:
```
import torch
import torch.nn as nn
import torch.optim as optim
```

Task 1: GPU/Device Agnostic Code

Goal: Write code that runs on CPU, CUDA, or MPS (Mac) automatically.

In [ ]:
```
device = 'cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is
print(f'Using device: {device}')


sample_input = torch.randn(1, 10).to(device)
type(sample_input)
```

Section 2: MNIST Project

Step 1: What is MNIST & Downloading Data

Concept: MNIST is the "Hello World" of Machine Learning. It contains 70,000 images of handwritten digits (0-9).

The Goal: Teach the computer to look at a grid of pixels and say "That is a 7".

The Data: Each image is grayscale and exactly 28×28

In [ ]:
```
from torchvision import datasets
from torchvision.transforms import ToTensor
```

In [ ]:
```
training_data = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)
```

```
# transform=ToTensor() converts the image (0-255) to a Torch Tensor (0.0-1.0)
```

In [ ]:
```
# Do for Testing
testing_data = datasets.MNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)
```

Visualizing One Image & Understanding Shapes

In [ ]:
```
len(training_data)
```

In [ ]:
```
len(testing_data)
```

In [ ]:
```
img,label = training_data[655]
```

In [ ]:
```
img.shape
```

In [ ]:
```
img.squeeze().shape
```

Display one Image in MatplotLib

In [ ]:
```
import matplotlib.pyplot as plt
```

In [ ]:
```
plt.imshow(img.squeeze(),cmap='gray')
plt.title(f"This is a number {label} ")
plt.show()
```

Calculation for the Input Layer

Concept: We are building a Linear (Feed-Forward) Network,

A Linear Layer consists of neurons in a single vertical line.

Our image is a square grid (28×28).

The Division: We must "cut" the image row by row and stack them into one long line.

The Calculation:

Height×Width=Total Input Features 28×28=784 So, our Input Layer must have 784 neurons.

In [ ]:
```
img, label = training_data[1]
```

In [ ]:
```
img.squeeze().shape
```

The Architecture (1 Input, 1 Hidden, 1 Output

The Concept: We will build the simplest standard network.

Input Layer (784): Receives the pixels.

Hidden Layer (128): The "brain" that learns shapes (loops, lines). We pick 128 because it's enough to learn but not too big.

Output Layer (10): The final decision. We have 10 digits (0-9), so we need 10 output scores.

In [ ]:
```
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()

        self.layers = nn.Sequential(
            nn.Linear(784,128),
            nn.ReLU(),
            nn.Linear(128,10)
        )
    def forward(self,x):
        x = self.flatten(x)
        logits = self.layers(x)
        return logits

model = SimpleNet()
print(model)
```

Understanding Batch Size (The Stack)

The Concept: Think of the model like a teacher grading exams.

Batch Size = 1: The teacher grades 1 exam, updates the grade book, then picks up the next exam. (Too slow).

Batch Size = 64: The teacher picks up a stack of 64 exams, grades them all at once, and updates the grade book one time for the whole stack. (Much faster).

We use DataLoader to create these "stacks" for us.

In [ ]:
```
from torch.utils.data import DataLoader

# Create stacks
train_loader = DataLoader(training_data, batch_size=64, shuffle=True)
# do for Test_data
test_loader = DataLoader(testing_data,batch_size=64,shuffle=True)

print(len(train_loader),"--",len(test_loader))
```

The Training Loop

In [ ]:
```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()
```

In [ ]:
```
def train():
    model.train()

    num_epochs = 5

    for epoch in range(num_epochs):
        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")

        for batch_No, (data, target) in enumerate(train_loader):

            output = model(data)

            loss = loss_fn(output, target)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            if batch_No % 100 == 0:
                print(f"Batch {batch_No}: Loss = {loss.item():.4f}")

        print(f"Epoch {epoch + 1} completed!")
```

In [ ]:
```
train()
```

In [ ]:
```
def test():
    model.eval()
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)

            prediction = output.argmax(dim=1)

            correct += (prediction == target).sum().item()

    accuracy = correct / len(test_loader.dataset)
    print(f"Test Accuracy: {accuracy:.1%}")

test()
```